

RAG Fundamentals: Chunking, Embeddings, and Vector Stores

Retrieval-Augmented Generation (RAG) is an architecture that enhances large language model (LLM) responses by retrieving relevant external knowledge at inference time. Instead of relying solely on parameters learned during training, a RAG system retrieves document chunks from a vector store and injects them into the LLM prompt as context. This allows the model to answer questions grounded in specific documents rather than general knowledge.

1. Document Chunking Strategies

Chunking is the process of splitting source documents into smaller pieces before embedding and indexing. The choice of chunking strategy directly affects retrieval quality. Chunks that are too large reduce precision; chunks that are too small lose context and coherence.

1.1 Fixed-Size Chunking

Fixed-size chunking splits text into segments of a predetermined token or character count, typically with an overlap to preserve cross-boundary context. A common configuration uses 512-token chunks with a 64-token overlap. This method is simple and fast but ignores sentence and paragraph boundaries, sometimes cutting mid-sentence.

1.2 Semantic Chunking

Semantic chunking splits documents based on meaning rather than token count. It uses embedding similarity between consecutive sentences to detect topic shifts: when the cosine similarity between adjacent sentences drops below a threshold, a chunk boundary is inserted. This produces semantically coherent chunks at the cost of variable chunk sizes, which can complicate batching.

1.3 Recursive Character Splitting

Recursive character splitting attempts to split on natural boundaries in a priority order: paragraphs first, then sentences, then words, then characters. LangChain's `RecursiveCharacterTextSplitter` implements this approach. It is the most commonly used chunking method in production RAG systems due to its balance between coherence and predictability.

1.4 Document-Aware Chunking

Document-aware chunking uses the structure of the source format. For PDFs, this may involve splitting by section headers detected via font size or bold formatting. For markdown, splitting by heading level is natural. This method produces the highest-quality chunks when document structure is consistent and

machine-readable.

2. Embedding Models

An embedding model converts a text chunk into a dense vector representation in a high-dimensional space. Semantically similar texts produce vectors with high cosine similarity. The embedding model is the most critical component for retrieval quality, as all similarity comparisons at query time depend on it.

2.1 Sentence Transformers

Sentence Transformers are bi-encoder models trained specifically to produce semantically meaningful sentence-level embeddings. The all-MiniLM-L6-v2 model produces 384-dimensional vectors and is widely used for its balance of speed and quality. Larger models such as all-mpnet-base-v2 produce 768-dimensional vectors with higher accuracy but slower inference.

2.2 OpenAI Embeddings

OpenAI's text-embedding-3-small and text-embedding-3-large models are API-based embedding services. text-embedding-3-small produces 1536-dimensional vectors and is optimized for cost-efficiency. text-embedding-3-large produces the same dimensionality with higher accuracy. These models are closed-source and require an API key, making them unsuitable for fully offline deployments.

2.3 Dimensionality and Distance Metrics

Embedding dimensionality affects both storage cost and retrieval accuracy. Higher-dimensional embeddings capture more semantic nuance but require more memory and slower approximate nearest-neighbor search. Cosine similarity is the standard distance metric for text embeddings, as it measures the angle between vectors independently of their magnitude. Dot product similarity is equivalent to cosine similarity when vectors are L2-normalized.

3. Vector Stores

A vector store is a database optimized for storing and querying high-dimensional embedding vectors. It supports approximate nearest-neighbor (ANN) search to find the top-K most similar vectors to a query embedding efficiently.

3.1 FAISS

FAISS (Facebook AI Similarity Search) is an open-source library for efficient similarity search in dense vector spaces. It runs entirely in memory and supports multiple index types: Flat (exact brute-force), IVF (inverted file, approximate), and HNSW (hierarchical navigable small world graphs). FAISS does

not support metadata filtering natively and is best suited for small-to-medium datasets that fit in RAM.

3.2 Qdrant

Qdrant is a purpose-built vector database with support for payload-based filtering, named collections, and multi-tenancy via payload fields. It supports HNSW indexing with configurable `m` and `ef_construction` parameters. Qdrant can be deployed locally via Docker or accessed through Qdrant Cloud. It supports filtering by metadata fields at query time, enabling scoped retrieval such as filtering by `user_id` or `session_id` without scanning irrelevant vectors.

3.3 Pinecone and Weaviate

Pinecone is a fully managed cloud-native vector database with automatic scaling. It supports namespaces for logical data separation. Weaviate is an open-source vector database that integrates directly with embedding models and supports hybrid search combining dense vector similarity with BM25 keyword search. Hybrid search improves recall on queries involving rare terms or exact identifiers that semantic search alone may miss.

4. The RAG Pipeline

A standard RAG pipeline consists of two phases: indexing and retrieval-augmented generation. During indexing, documents are chunked, embedded, and stored in a vector store with associated metadata. During inference, a user query is embedded using the same model, and the top-K most similar chunks are retrieved. These chunks are formatted into a prompt template and passed to an LLM to generate a grounded response.

The quality of the final response depends on three factors: retrieval precision (are the right chunks retrieved), context utilization (does the LLM use the retrieved context faithfully), and generation quality (does the LLM produce a coherent, accurate response). Failures in any of these three stages propagate to the final output.